

Probabilistic Robotics Course

EKF SLAM [Application]

Giorgio Grisetti

`grisetti@diag.uniroma1.it`

Department of Computer, Control and Management Engineering
Sapienza University of Rome



EKF SLAM: recap

- Estimate the current state distribution from
 - Previous state distribution
 - Sequence of observations $\mathbf{z}_{0:t}$
 - Sequence of controls $\mathbf{u}_{0:t-1}$
 - Transition model
 - Observation model

$$\begin{aligned}\mu_{t|t-1} &= \mathbf{f}(\mu_{t-1|t-1}, \mathbf{u}_{t-1}) \\ \Sigma_{t|t-1} &= \mathbf{A}_t \Sigma_{t-1|t-1} \mathbf{A}_t^T + \mathbf{B}_t \Sigma_u \mathbf{B}_t^T\end{aligned}$$

$$\mu_z = \mathbf{h}(\mu_{t|t-1})$$

$$\begin{aligned}\mu_{t|t} &= \mu_{t|t-1} + \mathbf{K}_t (\mathbf{z}_t - \mu_z) \\ \Sigma_{t|t} &= (\mathbf{I} - \mathbf{K}_t \mathbf{C}_t) \Sigma_{t|t-1}\end{aligned}$$

ekf_slam

File Edit Help



Z+

Z-



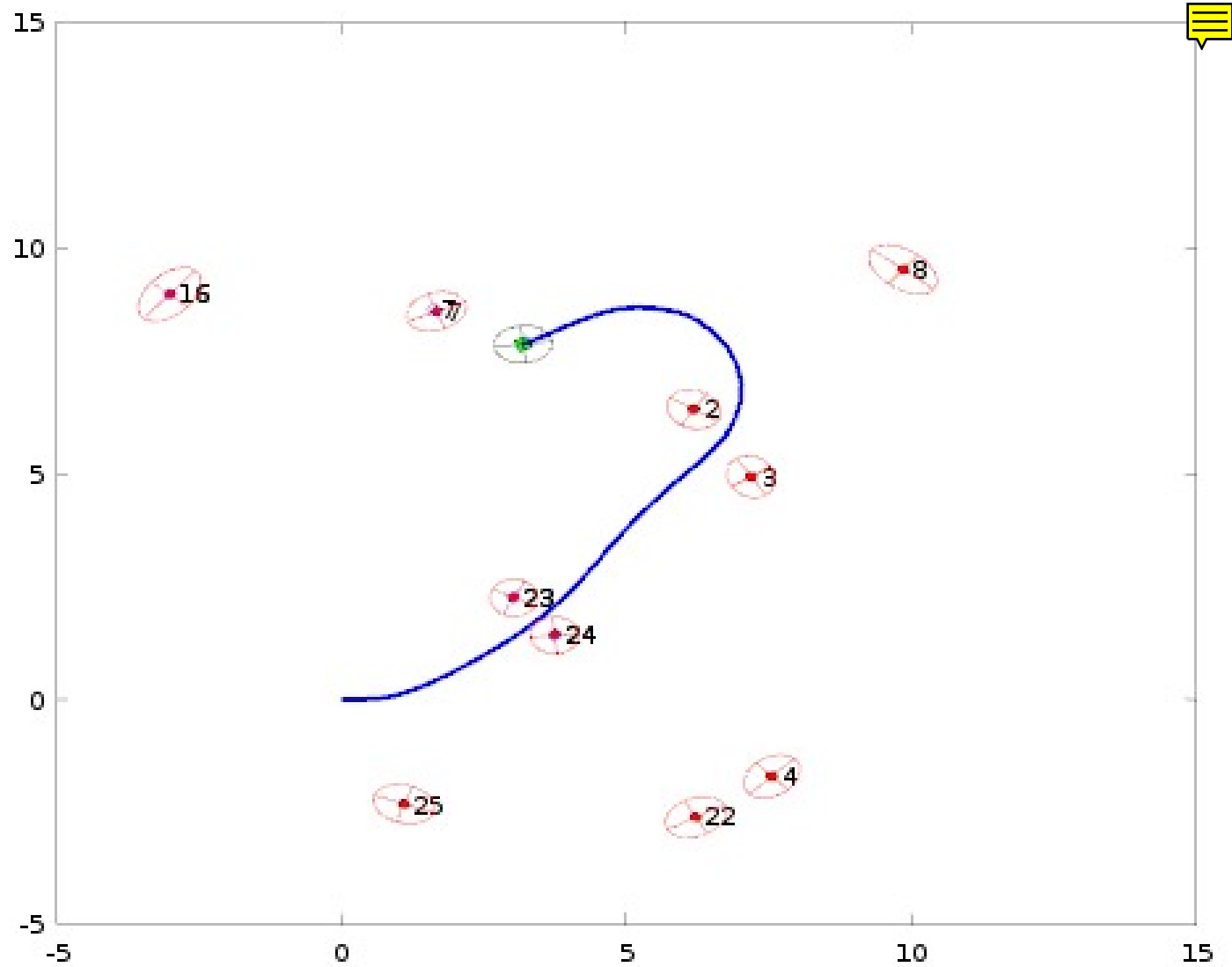
Insert Text



Axes

Grid

Autoscale



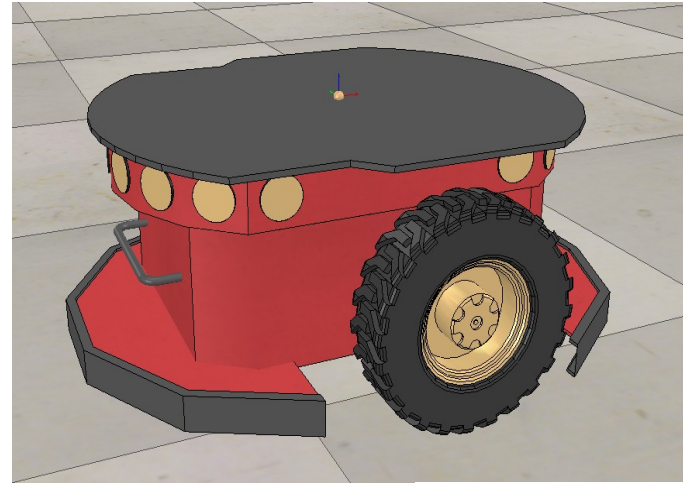
Outline

- Scenario
- Controls
- Observations
- Non-Linear Systems and Gaussian Noise
- Extended Kalman Filter SLAM

Scenario

Orazio moves on a 2D plane

- Is controlled by translational and rotational velocities
- Senses a set of uniquely distinguishable landmarks through a “2D landmark sensor”
- The location of the landmarks in the world is **not** known



Approaching the problem

We want to develop a KF based algorithm to track the position of Orazio as it moves (localization) and, at the same time, the position of the observed landmarks (mapping).

The inputs of our algorithms will be

- velocity measurements
- landmark measurements

We have no prior knowledge of the map!

EKF

1. Predict: incorporate new control

$$\begin{aligned}\mu_{t|t-1} &= \mathbf{f}(\mu_{t-1|t-1}, \mu_{u,t-1}) \\ \mathbf{A}_t &= \left. \frac{\partial \mathbf{f}(\mathbf{x}, \mathbf{u})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mu_{t-1|t-1}} \\ \mathbf{B}_t &= \left. \frac{\partial \mathbf{f}(\mathbf{x}, \mathbf{u})}{\partial \mathbf{u}} \right|_{\mathbf{u}=\mu_{u,t-1}} \\ \Sigma_{t|t-1} &= \mathbf{A}_t \Sigma_{t-1|t-1} \mathbf{A}_t^T + \mathbf{B}_t \Sigma_u \mathbf{B}_t^T\end{aligned}$$

2. Correct: incorporate new measurement

$$\begin{aligned}\mathbf{C}_t &= \left. \frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mu_{t|t-1}} \\ \mathbf{K}_t &= \Sigma_{t|t-1} \mathbf{C}_t^T (\Sigma_z + \mathbf{C}_t \Sigma_{t|t-1} \mathbf{C}_t^T)^{-1} \text{ 🗨️} \\ \mu_{t|t} &= \mu_{t|t-1} + \mathbf{K}_t (\mathbf{z}_t - \mathbf{h}(\mu_{t|t-1})) \\ \Sigma_{t|t} &= (\mathbf{I} - \mathbf{K}_t \mathbf{C}_t) \Sigma_{t|t-1}\end{aligned}$$

innovation

3. SLAM: add new landmarks to state

Domains: State Space

Robot:

$$\Rightarrow \mathbf{X}_t^{[r]} = [\mathbf{R}_t | \mathbf{t}_t] \in SE(2) \xrightarrow{\quad} \mathbf{x}_t^{[r]} = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix} \in \mathbb{R}^3$$

Landmarks:

$$\mathbf{x}_t^{[n]} = \begin{pmatrix} x_t^{[n]} \\ y_t^{[n]} \end{pmatrix} \in \mathbb{R}^2 \quad n=1,2,\dots,N$$

Full state vector:

$$\mathbf{x}_t = \begin{pmatrix} \mathbf{x}_t^{[r]} \\ \mathbf{x}_t^{[1]} \\ \mathbf{x}_t^{[2]} \\ \vdots \\ \mathbf{x}_t^{[N]} \end{pmatrix} \in \mathbb{R}^{(3+2N)}$$

Domains: Ctrl and Meas.

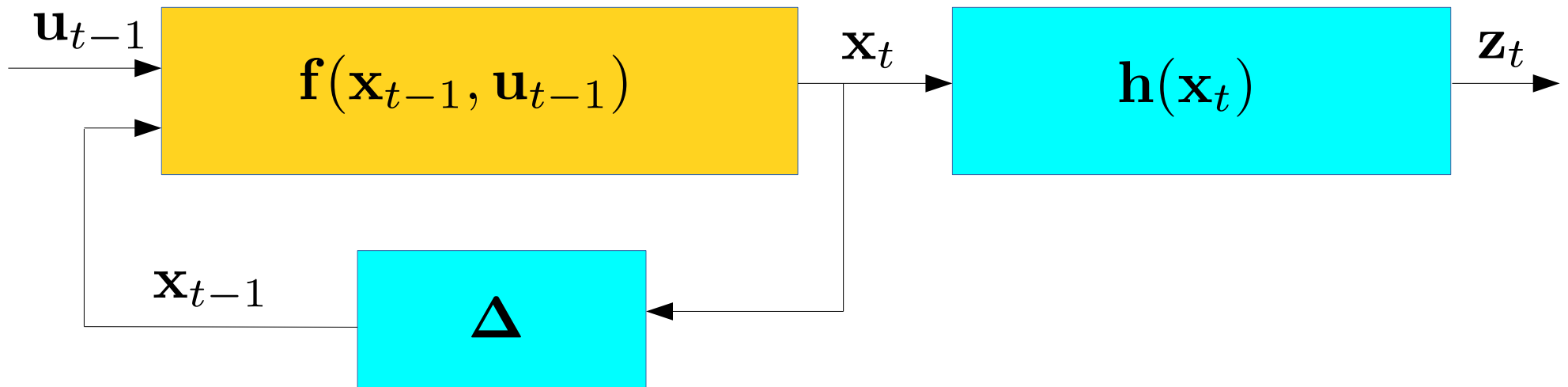
Controls: (same as in EKF Localization)

$$\mathbf{u}_t = \begin{pmatrix} u_t^{[1]} \\ u_t^{[2]} \end{pmatrix} \in \mathbb{R}^2$$

Measurements:

$$\mathbf{z}_t^{[m]} = \begin{pmatrix} x_t^{[m]} \\ y_t^{[m]} \end{pmatrix} \in \mathbb{R}^2 \quad m=1..M$$

Transition Function

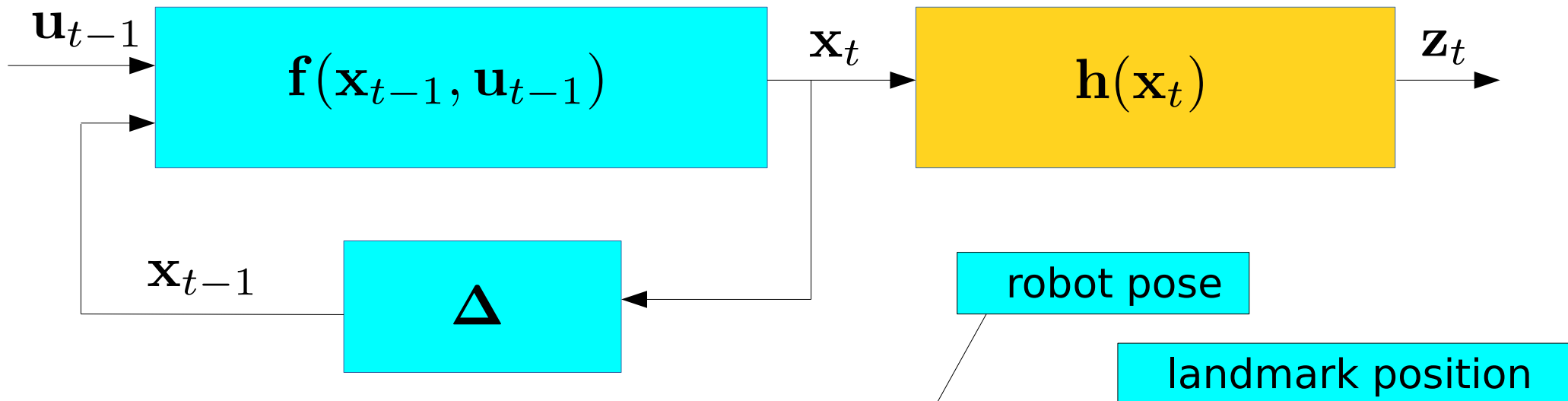


pose update (the robot moves)

the landmarks don't move

$$\mathbf{x}_t = \mathbf{f}(\mathbf{x}_{t-1}, \mathbf{u}_{t-1}) = \begin{pmatrix} x_{t-1} + u_{t-1}^{[1]} \cdot \cos(\theta_{t-1}) \\ y_{t-1} + u_{t-1}^{[1]} \cdot \sin(\theta_{t-1}) \\ \theta_{t-1} + u_{t-1}^{[2]} \\ \mathbf{x}_{t-1}^{[1]} \\ \mathbf{x}_{t-1}^{[2]} \\ \vdots \\ \mathbf{x}_{t-1}^{[N]} \end{pmatrix}$$

Measurement Function



$$\begin{aligned}
 \mathbf{z}_t^{[n]} &= \mathbf{h}^{[n]}(\mathbf{x}_t) = \mathbf{h}(\mathbf{x}_t^{[r]}, \mathbf{x}_t^{[n]}) \\
 &= \mathbf{R}_t^T (\mathbf{v}_t^{[n]} - \mathbf{t}_t) \\
 n = 1, \dots, N &= \begin{pmatrix} \cos \theta_t (x_t^{[n]} - x_t) + \sin \theta_t (y_t^{[n]} - y_t) \\ -\sin \theta_t (x_t^{[n]} - x_t) + \cos \theta_t (y_t^{[n]} - y_t) \end{pmatrix}
 \end{aligned}$$

We have N potential measurement functions, one for each landmark

relative position of the n^{th} landmark w.r.t. the robot at time t

Control Noise

We assume the control inputs are effected by a zero-mean Gaussian noise resulting from the sum of two aspects:

- a constant noise σ_u
- velocity dependent terms whose standard deviation grows with the speed (i.d.)
 - translational noise standard deviation: $\sigma_T = u_t^{[1]}$
 - rotational noise standard deviation: $\sigma_R = u_t^{[2]}$

$$\mathbf{n}_{u,t} \sim \mathcal{N}(\mathbf{n}_{u,t}; \mathbf{0}, \underbrace{\begin{pmatrix} \sigma_u^2 + \sigma_T^2 & 0 \\ 0 & \sigma_u^2 + \sigma_R^2 \end{pmatrix}}_{\Sigma_u})$$

Measurement Noise

For each landmark we measure, we consider a Gaussian noise (in x and y):

$$\mathbf{n}_z \sim \mathcal{N}(\mathbf{n}_z; \mathbf{0}, \underbrace{\begin{pmatrix} \sigma_z^2 & 0 \\ 0 & \sigma_z^2 \end{pmatrix}}_{\Sigma_z})$$

We assume it is zero mean and constant with standard deviation σ_z .

Jacobian 1: State

$$\text{f}(\mathbf{x}_{t-1}, \mathbf{u}_{t-1}) = \begin{pmatrix} x_{t-1} + u_{t-1}^{[1]} \cdot \cos(\theta_{t-1}) \\ y_{t-1} + u_{t-1}^{[1]} \cdot \sin(\theta_{t-1}) \\ \theta_{t-1} + u_{t-1}^{[2]} \\ \mathbf{x}_{t-1}^{[1]} \\ \mathbf{x}_{t-1}^{[2]} \\ \vdots \\ \mathbf{x}_{t-1}^{[N]} \end{pmatrix}$$

$$\mathbf{A}_t = \left. \frac{\partial \mathbf{f}(\cdot)}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mu_{t-1|t-1}} = \left(\begin{array}{cccc} \frac{\partial \mathbf{f}(\cdot)}{\partial \mathbf{x}^{[r]}} & \frac{\partial \mathbf{f}(\cdot)}{\partial \mathbf{x}^{[1]}} & \frac{\partial \mathbf{f}(\cdot)}{\partial \mathbf{x}^{[2]}} & \dots & \frac{\partial \mathbf{f}(\cdot)}{\partial \mathbf{x}^{[N]}} \end{array} \right)$$

$$\frac{\partial \mathbf{f}(\cdot)}{\partial \mathbf{x}^{[r]}} = \begin{pmatrix} 1 & 0 & -u_{t-1}^{[1]} \cdot \sin(\theta_{t-1}) \\ 0 & 1 & +u_{t-1}^{[1]} \cdot \cos(\theta_{t-1}) \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 0 \end{pmatrix} \quad \frac{\partial \mathbf{f}(\cdot)}{\partial \mathbf{x}^{[2]}} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \vdots & \vdots \\ 1 & 0 \\ 0 & 1 \\ \vdots & \vdots \\ 0 & 0 \end{pmatrix}$$

Jacobian 2: Controls

$$\mathbf{f}(\mathbf{x}_{t-1}, \mathbf{u}_{t-1}) = \begin{pmatrix} x_{t-1} + u_{t-1}^{[1]} \cdot \cos(\theta_{t-1}) \\ y_{t-1} + u_{t-1}^{[1]} \cdot \sin(\theta_{t-1}) \\ \theta_{t-1} + u_{t-1}^{[2]} \\ \mathbf{x}_{t-1}^{[1]} \\ \mathbf{x}_{t-1}^{[2]} \\ \vdots \\ \mathbf{x}_{t-1}^{[N]} \end{pmatrix}$$

$$\mathbf{B}_t = \left. \frac{\partial \mathbf{f}(\mathbf{x}, \mathbf{u})}{\partial \mathbf{u}} \right|_{\mathbf{u}=\mathbf{u}_{t-1}} = \begin{pmatrix} \begin{array}{cc} \cos(\theta_{t-1}) & 0 \\ \sin(\theta_{t-1}) & 0 \\ 0 & 1 \end{array} \\ \begin{array}{cc} 0 & 0 \\ 0 & 0 \end{array} \\ \vdots \\ \begin{array}{cc} 0 & 0 \end{array} \end{pmatrix}$$

pose block

←

landmark block 1

←

landmark block N

←

Jacobian 3: Measurements

For the measurement function, we need to derive w.r.t. **all** state variables (N+3)

Prediction of the n^{th} landmark observation:

$$\mathbf{h}^{[n]}(\mathbf{x}_t) = \mathbf{R}_t^T (\mathbf{x}_t^{[n]} - \mathbf{t}_t)$$

The n^{th} measurement **Jacobian** will have a block structure

$$\mathbf{C}_t^{[n]} = \left. \frac{\partial \mathbf{h}^{[n]}(\cdot)}{\partial \mathbf{x}} \right|_{x=\mu_{t|t-1}} = \left(\begin{array}{ccccccc} \frac{\partial \mathbf{h}^{[n]}}{\partial \mathbf{x}^{[r]}} & \mathbf{0} & \dots & \frac{\partial \mathbf{h}^{[n]}}{\partial \mathbf{x}^{[n]}} & \dots & \mathbf{0} \end{array} \right)$$

Jacobian 3: Measurements

We need to calculate the derivatives only for the non zero blocks

$$\mathbf{h}^{[n]}(\mathbf{x}_t) = \mathbf{R}_t^T (\mathbf{x}_t^{[n]} - \mathbf{t}_t)$$

$$\frac{\partial \mathbf{h}^{[n]}(\cdot)}{\partial \mathbf{x}^{[r]}} = \left(-\mathbf{R}_t^T \quad \frac{\partial \mathbf{R}_t^T}{\partial \theta_t} (\mathbf{x}_t^{[n]} - \mathbf{t}_t) \right)$$

$$\frac{\partial \mathbf{h}^{[n]}(\cdot)}{\partial \mathbf{x}^{[n]}} = \mathbf{R}_t^T$$

$$\mathbf{C}_t^{[n]} = \left. \frac{\partial \mathbf{h}^{[n]}(\cdot)}{\partial \mathbf{x}} \right|_{x=\mu_{t|t-1}} = \left(\begin{array}{ccc} \frac{\partial \mathbf{h}^{[n]}}{\partial \mathbf{x}^{[r]}} & \mathbf{0} & \dots & \frac{\partial \mathbf{h}^{[n]}}{\partial \mathbf{x}^{[n]}} & \dots & \mathbf{0} \end{array} \right)$$

pose block

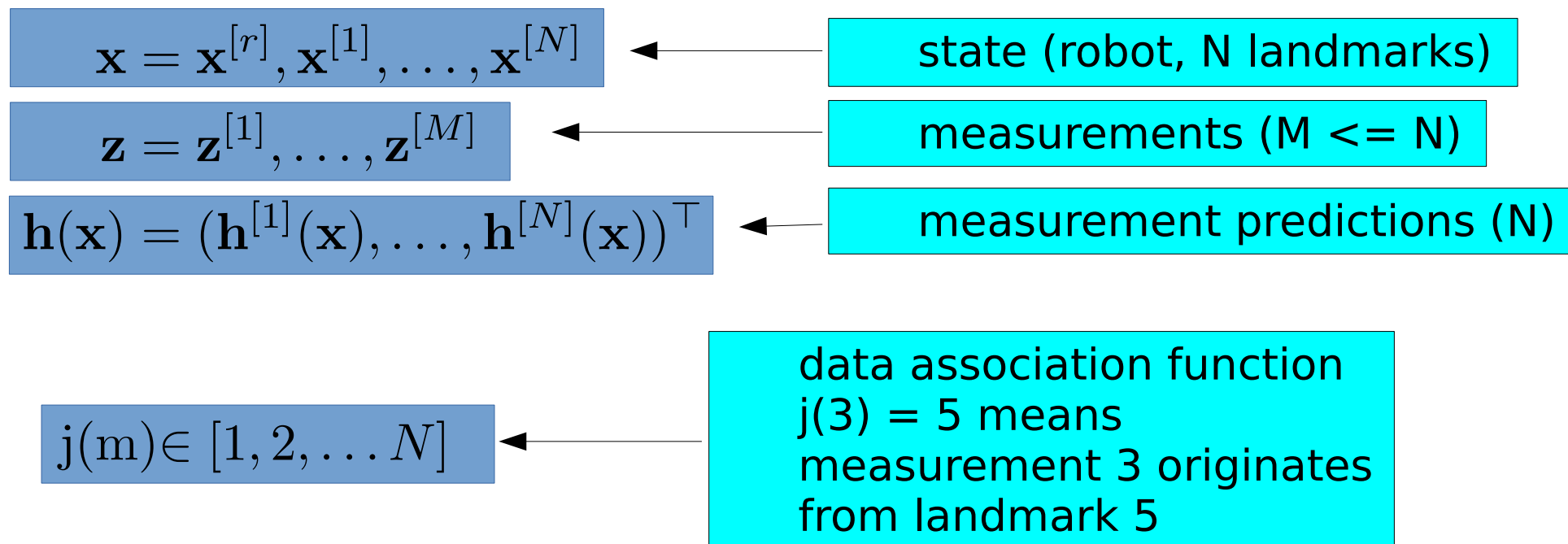
landmark block

Data Association

The observation z_t will originate from a subset of landmarks in the state.

The order of these measurements does not necessarily match the order of the landmarks in the state.

Let $j(m)$ be the index of the landmark that generates the m^{th} measurement block.



Data Association

For now we assume to know the assignment $j(m)$.

With this assignment we can build a prediction based on M measured landmarks, and its Jacobian!

$$\mathbf{h}(\mathbf{x}_t) = \begin{pmatrix} \mathbf{h}^{[j(1)]} \\ \mathbf{h}^{[j(2)]} \\ \vdots \\ \mathbf{h}^{[j(M)]} \end{pmatrix} \quad \mathbf{C}_t = \frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial \mathbf{h}^{[j(1)]}}{\partial \mathbf{x}} \\ \frac{\partial \mathbf{h}^{[j(2)]}}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial \mathbf{h}^{[j(M)]}}{\partial \mathbf{x}} \end{pmatrix}$$

Populating the Map

Time: t

current state

$$\mu_t = \begin{pmatrix} \mathbf{x}_t^{[r]} \end{pmatrix}$$

$$\mathbf{x}_t^{[r]} = \begin{pmatrix} x_t \\ y_t \\ \theta_t \end{pmatrix}$$

robot state

Id: 1



Id: 7



Id: 9



$$\text{id_to_state_map} = \begin{pmatrix} -1 & -1 & \dots & \dots & -1 \end{pmatrix}$$

$$\text{state_to_id_map} = \begin{pmatrix} -1 & -1 & \dots & \dots & -1 \end{pmatrix}$$

Populating the Map

Id: 1



Id: 7



Id: 9



Time: $t+1$
current state

$$\mu_{t+1} = \begin{pmatrix} \mathbf{x}_{t+1}^{[r]} \\ \mathbf{x}_{t+1}^{[7]} \end{pmatrix}$$

$$\mathbf{x}_t^{[7]} = \begin{pmatrix} x_t^{[7]} \\ y_t^{[7]} \end{pmatrix}$$

landmark state

Position 7

$$\text{id_to_state_map} = \begin{pmatrix} -1 & \dots & 1 & -1 & \dots & -1 \end{pmatrix}$$

$$\text{state_to_id_map} = \begin{pmatrix} 7 & -1 & \dots & \dots & -1 \end{pmatrix}$$

Populating the Map

Time: $t+2$
current state

$$\mu_{t+2} = \begin{pmatrix} \mathbf{x}_{t+2}^{[r]} \\ \mathbf{x}_{t+2}^{[7]} \\ \mathbf{x}_{t+2}^{[9]} \end{pmatrix}$$

$$\mathbf{x}_t^{[9]} = \begin{pmatrix} x_t^{[9]} \\ y_t^{[9]} \end{pmatrix}$$

Id: 1



Id: 7



Id: 9



Position 9

$$\text{id_to_state_map} = \begin{pmatrix} -1 & \dots & 1 & -1 & 2 & \dots & -1 \end{pmatrix}$$

$$\text{state_to_id_map} = \begin{pmatrix} 7 & 9 & -1 & \dots & \dots & -1 \end{pmatrix}$$

Populating the Map

Time: $t+3$
current state

$$\mu_{t+3} = \begin{pmatrix} \mathbf{x}_{t+3}^{[r]} \\ \mathbf{x}_{t+3}^{[7]} \\ \mathbf{x}_{t+3}^{[9]} \\ \mathbf{x}_{t+3}^{[1]} \end{pmatrix}$$

Id: 1



Id: 7



Id: 9



$$\text{id_to_state_map} = \begin{pmatrix} \text{3} & \dots & \text{1} & -1 & \text{2} & \dots & -1 \end{pmatrix}$$

$$\text{state_to_id_map} = \begin{pmatrix} \text{7} & \text{9} & \text{1} & \dots & \dots & -1 \end{pmatrix}$$

Updating the Map

Partition the sensed landmarks in two classes:

- Already known landmarks (part of the state)
Use them to perform an EKF correction
- New landmarks
Add them to the state **after** the EKF correction

Hands On!

g2o Wrapper

Load your V-REP acquired dataset

```
[land, pose, transition, obs] = loadG2o('my_dataset.g2o');
```

This call returns an array of objects with the fields:

This time we don't know the landmarks!

pose =

1x137 struct array containing the fields:

id
x
y
theta

transition =

1x136 struct array containing the fields:

id_from
id_to
v

obs =

1x136 struct array containing the fields:

pose_id
observation